

Module 7

Real-World Verification Applications & VIP (SV/UVM)

Learning objectives

- Apply the full methodology (planning, UVM env, coverage, regressions) to **real-world style blocks...

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["End-to-End IP Verification"]

T2["VIP Design and Reuse"]

T1 --> T2

T3["Production Quality and Maint"]

T2 --> T3

Apply the full methodology (planning, UVM env, coverage, regressions) to **real-world style blocks and protocols** (e.g., DMA, UART/SPI/I2C...

Overview

- Modules 1–6 built up your **skills** and infrastructure. Module 7 is about **applying** them to rea...

How to learn this module

Suggested learning path

- Choose a real-world DUT or subsystem scope in
`module7/PROJECTS.md`.
- Scaffold Module 7 workspace: `./scripts/module7.sh
--scaffold`
- Design reusable VIP structure in
`module7/VIP\_DESIGN.md`.
- Apply best practices from
`module7/BEST\_PRACTICES.md` to naming, docs, and API
boundaries.
- Validate: `./scripts/module7.sh --check`

Design architecture

1. Real-world DUT / subsystem

- Choose DMA-style block,
bus/register IP, or protocol DUT
(UART/SPI/I2C/AXI-lite style).
- Document block diagram,
clocks/resets, and interface
contracts in `PROJECTS.md`.

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

subgraph DUT["stream_fifo (common_dut/rtl)"]

SRC["Source interface\ns_valid / s_ready / s_data"]

MEM["Circular buffer\nmem[DEPTH], wr_ptr, rd_ptr"]

SNK["Sink interface\nm_valid / m_ready / m_data"]

STS["Status\nlevel, overflow, underflow"]

2. VIP package architecture

- Reusable VIP: agent, sequencer, driver, monitor, checker, coverage, config objects.
- `VIP\_DESIGN.md` defines public API, sequences, and integration guide for adopters.
- Package boundaries: what integrators configure vs what VIP owns internally.

```
Verification Architecture
Diagram: Verification Architecture
(render with mmdc when available)
flowchart TB
    subgraph VIP["Reusable VIP package"]
        AGT["Agent"]
        CHK["Checker"]
        COV["Coverage"]
        DOC["Documentation"]
    end
```

3. Production layout

- Separate ``rtl/``, ``tb/``, ``tests/``, ``docs/`` with consistent naming for handoff to another team.

VIP design template

```
# Verification IP (VIP) Design Plan (Module 7)
```

```
> **Purpose**: Design and document a reusable Verification IP (VIP) for  
a chosen protocol or IP block.
```

```
> **Module**: 7 – Real-World Verification Applications & VIP (SV/UVM)
```

```
> **Instructions**: Fill in this document for your design. Copy from  
`module7/templates/` if needed, or edit the file in `module7/`  
directly.
```

```
Reference: `module7/.solutions/`.
```

1. VIP Overview

```
<!-- TODO: Protocol/IP name, intended use. What this VIP should enable  
users to do (drive, observe, check, measure, configure). -->
```

2. VIP Components

Key files to study

Open these in the repo

- ``module7/PROJECTS.md`` — DUT scope, interfaces, and verification goals
- ``module7/VIP_DESIGN.md`` — VIP package architecture and public API
- ``module7/BEST_PRACTICES.md`` — production-quality verification and delivery standards
- ``module6/MULTI_AGENT_ARCHITECTURE.md`` — prior multi-agent patterns to reuse
- ``scripts/module7.sh`` — project and VIP design doc checks

Verification & testing methods

1. End-to-end IP verification

- Full lifecycle: requirements →
env → tests → coverage →
regressions → sign-off preview.
- Regression tiers and coverage
goals match production
expectations.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TD

PLAN["Project test plan"] --> ENV["Full UVM env"]

ENV --> REG["Production regressions"]

REG --> CLOSE["Coverage closure"]

CLOSE --> SIGN["Sign-off criteria"]

BP["BEST_PRACTICES.md"] --> ENV

2. VIP validation

- Self-test environment for the VIP plus integrator tests on a reference DUT.
- Documentation-driven review: another engineer can adopt VIP from docs alone.

3. Quality bar

- ``BEST_PRACTICES.md`` — coding standards, review checklist, maintenance and versioning.
- ``./scripts/module7.sh --check`` validates project and VIP design documentation.

Project scope template

```
# Real-World Verification Projects (Module 7)
```

```
> **Purpose**: Describe one or more real-world style verification projects you will implement (DMA, protocols, custom IP, etc.).
```

```
> **Module**: 7 – Real-World Verification Applications & VIP (SV/UVM)
```

```
> **Instructions**: Fill in this document for your design. Copy from `module7/templates/` if needed, or edit the file in `module7/` directly.
```

```
Reference: `module7/.solutions/`.
```

```
## 1. Project Overview
```

```
<!-- TODO: For each project: DUT/IP, scope (level, features), goals, status. Add more projects as needed. -->
```

```
## 2. Requirements and Use Cases
```

VIP best practices excerpt

```
# Best Practices & Quality Checklist (Module 7)
```

```
> **Purpose**: Capture and enforce best practices for code, structure,  
and maintenance in your real-world verification/VIP projects.
```

```
> **Module**: 7 – Real-World Verification Applications & VIP (SV/UVM)
```

```
> **Instructions**: Fill in this document for your design. Copy from  
`module7/templates/` if needed, or edit the file in `module7/`  
directly.
```

```
Reference: `module7/.solutions/`.
```

1. Project Structure

```
<!-- TODO: Guidelines: separation (DUT, UVM/VIP, tests/scripts, docs).  
Project-specific layout and naming. -->
```

2. Code Quality and Naming

Syllabus topics

1. End-to-End IP Verification

- Applying planning + UVM + coverage + regression to realistic IP.
- Realistic test planning, coverage closure, and regression operation.

2. VIP Design and Reuse

- Designing and documenting verification IP (VIP) for protocols or blocks.

3. Production Quality and Maintenance

- Best practices for project structure, code quality, docs, and maintenance.

Command reference highlights

Scaffold and validate Module 7

- `./scripts/module7.sh --scaffold`

Review VIP and project templates

- `head -40 module7/templates/VIP_DESIGN.md`

Hands-on examples

Module 7 self-check

```
$ ./scripts/module7.sh --help
```

Terminal: module7_check

```
███[0;36m=====███[0m
███[0;36mModule 7: Real-World Verification Applications & VIP███[0m
███[0;36m=====███[0m
```

Usage: ./scripts/module7.sh [OPTIONS]

Module 7: Real-World Verification Applications & VIP (SV/UVM)
This script summarizes and checks the planning artifacts for Module 7.

OPTIONS:

--projects-status	Show status of projects.md only
--vip-status	Show status of vip_design.md only
--best-practices	Show status of best_practices.md only
--checklist-status	Show status of checklist_module7.md only
--summary	Show all statuses (default)
--check	Media/CI self-check (structure only)
--demo	Print example commands from EXAMPLES.md
--scaffold	Copy templates/*.md into module7/ if missing
--help, -h	Show this help message

EXAMPLES:

```
# Full summary
./scripts/module7.sh

# Focus only on VIP design
./scripts/module7.sh --vip-status
```

Demo: VIP design template

```
$ head -30 module7/templates/VIP_DESIGN.md
```

Terminal: ex01_vip_design

```
# Verification IP (VIP) Design Plan (Module 7)
```

```
> **Purpose**: Design and document a reusable Verification IP (VIP) for a chosen protocol or IP
```

```
> **Module**: 7 – Real-World Verification Applications & VIP (SV/UVM)
```

```
> **Instructions**: Fill in this document for your design. Copy from `module7/templates/` if nee
```

```
## 1. VIP Overview
```

```
<!-- TODO: Protocol/IP name, intended use. What this VIP should enable users to do (drive, obser
```

```
## 2. VIP Components
```

```
<!-- TODO: Table of Component ID, Type, Purpose. Location in codebase. -->
```

```
## 3. Interfaces and Transactions
```

```
### 3.1 DUT Interface(s)
```

```
<!-- TODO: Clock/reset, control/data/status, handshakes. -->
```

```
### 3.2 Transaction Definition
```

```
<!-- TODO: Table of Transaction Class, Fields, Notes. Constraints and helpers. -->
```

```
## 4. Protocol Checks and Scoreboarding
```

```
### 4.1 Protocol Rules
```

```
<!-- TODO: Table of Rule ID, Description, Enforced By. Can reference module6/PROTOCOL_VERIFICATI
```

Demo: Projects overview

```
$ head -25 module7/templates/PROJECTS.md
```

Terminal: ex02_projects

```
# Real-World Verification Projects (Module 7)
```

```
> **Purpose**: Describe one or more real-world style verification projects you will implement (D  
> **Module**: 7 – Real-World Verification Applications & VIP (SV/UVM)
```

```
> **Instructions**: Fill in this document for your design. Copy from `module7/templates/` if nee
```

```
## 1. Project Overview
```

```
<!-- TODO: For each project: DUT/IP, scope (level, features), goals, status. Add more projects a
```

```
## 2. Requirements and Use Cases
```

```
<!-- TODO: For each project, list key requirements and use cases. Link to module1/REQUIREMENTS_M
```

```
## 3. Planned Verification Approach
```

```
<!-- TODO: For each project: Environment, tests (link to test_plan.md), coverage, regressions. D
```

```
## 4. Risks and Constraints
```

```
<!-- TODO: Risks, constraints, mitigations. -->
```

```
## 5. Milestones
```

Demo: Best practices

```
$ head -25 module7/templates/BEST_PRACTICES.md
```

Terminal: ex03_best_practices

```
# Best Practices & Quality Checklist (Module 7)
```

```
> **Purpose**: Capture and enforce best practices for code, structure, and maintenance in your r  
> **Module**: 7 – Real-World Verification Applications & VIP (SV/UVM)
```

```
> **Instructions**: Fill in this document for your design. Copy from `module7/templates/` if nee
```

```
## 1. Project Structure
```

```
<!-- TODO: Guidelines: separation (DUT, UVM/VIP, tests/scripts, docs). Project-specific layout a
```

```
## 2. Code Quality and Naming
```

```
<!-- TODO: Checklist: clear names, no magic numbers, consistent patterns, comments (why not just
```

```
## 3. Documentation
```

```
<!-- TODO: Checklist: component descriptions, public API docs, README (build, run, extend). Docs
```

```
## 4. Testing and Coverage Practices
```

```
<!-- TODO: Checklist: bug fix → test, test names reflect intent, tiers, coverage drives improvem
```

```
## 5. Regression and CI Hygiene
```


Demo: Reference VIP design

```
$ head -20 module7/.solutions/VIP_DESIGN.md
```

Terminal: ex04_solutions

Verification IP (VIP) Design Plan (Module 7)

> **Purpose**: Design and document a reusable Verification IP (VIP) for a chosen protocol or IP
> **Module**: 7 – Real-World Verification Applications & VIP (SV/UVM)

1. VIP Overview

- **Protocol / IP**: Ready/valid streaming protocol — single-beat handshake (valid && ready) for
- **Intended Use**:
 - Block-level verification (stream_fifo and similar DUTs).
 - Subsystem/system integration (reuse stream_if and agents where streaming interfaces appear).
 - Internal reuse first; can be packaged for wider reuse with documentation.

Summarize what this VIP should enable users to do:

- **Drive** source-side traffic (push) via UVM driver and sequences.
- **Observe** sink-side traffic (pop) via UVM monitor and analysis port.
- **Check** in-order correctness via scoreboard and protocol rules (overflow/underflow) via asse
- **Measure** functional coverage (operations, level, flags, backpressure).
- **Configure** virtual interfaces, seed, coverage enable; integrate into any env that provides

Demo: Module validation

```
$ ./scripts/module7.sh --check
```

Terminal: ex05_validate

```
███[0;36m=====███[0m
███[0;36mModule 7: Real-World Verification Applications & VIP███[0m
███[0;36m=====███[0m
```

Module 7: media self-check
OK: module7/ exists
OK: templates/ exists
OK: EXAMPLES.md exists
OK: docs/MODULE7.md exists
OK: common_dut/rtl/ exists

All required checks passed.

Practice & assessment

What you should know (1/2)

- Design and implement a complete verification environment for a realistic DUT.
- Create and document reusable VIP that others could adopt.
- Operate regressions and coverage closure as you would in a real team.
- Maintain production-quality code, docs, and tests over the lifetime of a project.
- At least one substantial real-world style project is described and partially/fully implemented.
- VIP or environment design is documented in ``module7/VIP_DESIGN.md`` and ``module7/PROJECTS.md``.

What you should know (2/2)

- Best-practices checklist largely satisfied
(`module7/BEST\_PRACTICES.md` /
`module7/CHECKLIST.md`).
- You can explain your project and design decisions as if presenting to a review board.

Assessment checklist

- At least one substantial real-world style project is described and partially/fully implemented.
- VIP or environment design is documented in ``module7/VIP_DESIGN.md`` and ``module7/PROJECTS.md``.
- Best-practices checklist largely satisfied (``module7/BEST_PRACTICES.md`` / ``module7/CHECKLIST.md``).
- You can explain your project and design decisions as if presenting to a review board.

Summary & next steps

- Apply the full methodology (planning, UVM env, coverage, regressions) to ****real-world style blocks...**
- Complete module7/CHECKLIST.md
- Review module7/EXAMPLES.md and run each lab
- Next: Next module in course